Operációs rendszerek függvénytár

Standard C – (dinamikus) memóriakezelés

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

void*	stdlib.h	malloc(size_t size)	*size* bájtnyi memóriát foglal le a heapen. **Mindig** ellenőrizd le a visszatérési értékét (nem NULL)! **Szabadítsd fel** a területet, ha már nem használod!	char* heap_ptr;
				if((heap_ptr = **malloc**(4)) != NULL){
				strcpy(heap_ptr, "abc");
				free(heap_ptr);
				}

void	stdlib.h	free(void* ptr)	Korábban malloctól vagy realloctól kapott pointert átadva szabadít fel heap memóriát.	

void*	stdlib.h	realloc(void* ptr,
		size_t size)
			Korábban heapen lefoglalt memóriaterület méretét változtatja *size*-ra (ha *ptr* NULL pointer, úgy viselkedik, mint malloc). Az adatok új címét adja vissza, amennyiben nem történt hiba. **Itt is** ellenőrizd le a visszatérési értéket! A **régi memóriacímet ne használd**, ha sikerült az újrafoglalás!	void* heap_ptr = malloc(2);
				if(heap_ptr != NULL){
				void* tmp_alloc = **realloc**(heap_ptr, 5);
				if(tmp_alloc != NULL){
				heap_ptr = tmp_alloc; //már 5 B lefoglalva
				}
				free(heap_ptr);
				}

void*	string.h	memcpy(void* s1,
		void* s2,
		size_t n)
			s2 címtől kezdve *n* bájtot másol át *s1* címtől kezdődő helyre. A cél-és forrás memóriatartományok között **nem** lehet átfedés! Visszaadja *s1* paramétert.	struct person{
				int age, weight;
				};
				int main(){
				struct person pr = {.age = 20, .weight = 75};
				struct person copy_of;
				memcpy(&copy_of, &pr, sizeof(struct person));
				return 0;
				}

void*	string.h	memmove(void* s1,
		void* s2,
		size_t n)
			s2 címtől kezdve *n* bájtot másol át *s1* címtől kezdődő helyre. A cél-és forrás memóriatartományok között **lehet** átfedés. Visszaadja *s1* paramétert.	int cons_yrs[] = {2019, 2020, 2020, 2021, 2022};
				memmove(&cons_yrs[1], &cons_yrs[2], 3 * sizeof(int)); //
				sizeof(int) == sizeof(*cons_yrs)
				cons_yrs[4] = 2023;

Fekete Máté

int	string.h	memcmp(void* s1,
		void* s2,
		size_t n)
			s1-ben és *s2*-ben megadott címektől kezdve *n* bájtot hasonlít össze. Visszatérési értéke 0, ha a két tartomány megegyezik, pozitív, ha a legelső eltérésnél (x+1. bájt) *s1*[x] > *s2*[x], negatív, ha fordítva.	int d1[] = {2017, 3, 30};
				int d2[] = {2017, 3, 30};
				if(**memcmp**(&d1, &d2, sizeof(d1)) == 0){
				//a két dátum megegyezik
				}

Standard C – karakterláncok kezelése

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

int	string.h	strcmp(char* s1,
		char* s2)
			2 NULL-terminált karakterlánc tartalmát hasonlítja össze. Működése hasonló a memcmp függvényéhez, viszont itt nem kell megadni, hogy hány bájtot hasonlítson össze (mivel a NULL terminátornál megáll).	char* str1_ptr = "AEZAKMI";
				char* str2_ptr = "AIWPRTON";
				if(**strcmp**(str1_ptr, str2_ptr) != 0){
				//a két szöveg nem egyezik meg
				}

size_t	string.h	strlen(char* s)	Megszámolja hány nem NULL bájt van *s*-től kezdődően egészen a legelső NULL bájtig.	char* str_ptr = "Cloudflare";
				size_t occupied_memory = **strlen**(str_ptr) + 1;
				// occupied_memory == 11

char*	string.h	strcat(char* dst, char* src)	Az első paraméterben megadott karakterlánc végére fűzi a másodikat. Figyelj arra, hogy legyen elég **rendelkezésre álló memóriaterület** *dst* után! *dst*-t adja vissza.	char buffer[50] = "John"; char* middle = " Fitzgerald"; char* last = " Kennedy"; **strcat**(buffer, middle); **strcat**(buffer, last); //John Fitzgerald Kennedy
char*	string.h	strcpy(char* dst, char* src)	A *src* által mutatott karakterláncot másolja át a NULL terminátorával együtt *dst*-től kezdődő helyre. Figyelj arra, hogy legyen elég **rendelkezésre álló memóriaterület** *dst*-től kezdve a karakterlánc eltárolásához! *dst*-t adja vissza.	char buffer[50]; **strcpy**(&buffer[0], "John"); **strcpy**(&buffer[4], " Fitzgerald"); **strcpy**(&buffer[15], " Kennedy");

Standard C, POSIX – fájlkezelés

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

FILE*	stdio.h (errno.h)	fopen(char* pathname, char* mode)	Megnyitja az első paraméterben megadott fájlt, a második paraméterben megadott módon. **Hiba esetén NULL** pointerrel tér vissza, ekkor errno értéke lesz a hibakód.	FILE* f = **fopen**("myfile.txt", "rb");
				if(f == NULL){
				printf("error reason code: %d\n", errno);
				}else{
				fclose(f);
				}

int	stdio.h	fclose(FILE* stream)	Bezárja a fájlt. Visszatérési értéke alapból 0, hiba esetén -1.	

size_t	stdio.h	fread(void* ptr,
		size_t size,
		size_t nmemb,
		FILE* stream)
			Legfeljebb *nmemb* darab, *size* méretű elemet olvas be a *ptr*-től kezdődő memóriaterületre a *stream* által azonosított fájlból. A beolvasott elemek számát adja vissza. Ha ez az érték kisebb, mint a kért elemszám, akkor vagy hiba történt vagy hamarabb elérte a fájl végét.	#define min(x, y) (((x) < (y)) ? (x) : (y))
				struct person people[20];
				size_t number_of_people;
				FILE* f = fopen("myfile.txt", "rb");
				if(f){
				if(**fread**(&number_of_people, sizeof(size_t), 1, f) == 1){
				fread(people, sizeof(struct person), min(20, number_of_people), f);
				}
				fclose(f);
				}

size_t	stdio.h	fwrite(void* ptr,
		size_t size,
		size_t nitems,
		FILE* stream)
			nitems darab, *size* méretű elemet ír ki a *ptr*-től kezdődő memóriaterületről a *stream* által azonosított fájlba. Visszaadja a sikeresen kiírt elemek számát, ami hiba esetén kisebb, mint *nitems*.	struct person{
				int age, weight;
				};
				int main(){
				struct person people[2] = {{...}, {...}};
				size_t number_of_people = 2;
				FILE* f = fopen("myfile.txt", "wb");
				if(f){
				if(**fwrite**(&number_of_people, sizeof(size_t), 1, f) == 1){
				fwrite(people, sizeof(struct person), number_of_people, f);
				}
				fclose(f);
				}
				return 0;
				}

Fekete Máté

int	stdio.h	fseek(FILE* stream,
		long offset,
		int whence)
			A *stream* fájl "kurzorát" helyezi át *whence*-hez képest *offset* bátjnyival jobbra. A *whence* SEEK_SET (fájl eleje), SEEK_CUR (jelenlegi hely), SEEK_END (fájl vége) értékeket, míg *offset* tetszőleges előjelű egész számértéket kaphat. 0-val tér vissza, ha nem történt hiba.	FILE* f = fopen("myfile.txt", "rb");
				long file_length;
				void* files_contents;
				if(f){
				fseek(f, 0, SEEK_END);
				file_length = **ftell**(f);
				if(file_length > 0){
				files_contents = malloc(file_length);
				if(files_contents){
				fseek(f, 0, SEEK_SET);
				if(fread(files_contents, file_length, 1, f) == 1){
				printf("👍\n");
				}
				free(files_contents);
				}
				}
				fclose(f);
				}

long int	stdio.h	ftell(FILE* stream)	Visszaadja, hogy hanyadik bájtnál áll *stream* "kurzora". Hiba esetén -1-et ad vissza.	

int	**fcntl.h**	open(char* pathname,
		int flags)
			A *pathname* elérési úton talált fájlt nyitja meg a második paraméterben megadott módon. Létrehozáskor a harmadik paraméterként megadandó, hogy ki milyen jogosultságokat kapjon a fájlhoz. Visszatér a (nemnegatív) fájlleíró számmal, vagy hiba esetén -1-gyel.	int file_to_display = **open**("myfile.txt", O_RDWR | O_CREAT, S_IRWXU | S_IRWXG | S_IRWXO);
				if(file_to_display < 0){
				perror("open error: ");
				}else{
				close(file_to_display);
				}

int	unistd.h	close(int fd)	Bezár egy fájlleírót. Visszatérési értéke alapból 0, hiba esetén -1.	

ssize_t	unistd.h	write(int fd,
		void* buf,
		size_t count)
			Legfeljebb *count* bájtot ír ki a *buf*-tól kezdődő memóriaterületről a *fd* által azonított fájlba. Visszaadja a ténylegesen kiírt bájtok számát vagy hiba esetén -1-et.	int logfile_fd = open("logfile.txt", O_WRONLY | O_CREAT | O_APPEND, S_IRUSR | S_IWUSR);
				char log[] = "[21:29:59] [Client thread/INFO]: GTNHLib version 0.0.13 loaded.\n";
				if(logfile_fd >= 0){
				ssize_t remaining = strlen(log);
				char *write_cur = log;
				while(remaining > 0){

				ssize_t temp_ret = **write**(logfile_fd, write_cur, remaining);
				if(temp_ret <= 0){
				break;
				}
				write_cur += temp_ret; //**write_cur nem véletlenül char***
				remaining -= temp_ret;
				}
				close(logfile_fd);
				}

ssize_t	unistd.h	read(int fd,
		void* buf,
		size_t count)
			Legfeljebb *count* bájtot olvas be *buf*-tól kezdődő memóriaterületre a *fd* fájlból. Visszaadja a ténylegesen beolvasott bájtok számát. A fájl végének elérése után 0-val, hiba esetén -1-gyel tér vissza.	int logfile_fd = open("logfile.txt", O_RDONLY);
				if (logfile_fd >= 0){
				char buf_char;
				while (**read**(logfile_fd, &buf_char, sizeof(buf_char)) > 0){
				printf("%c", buf_char);
				}
				close(logfile_fd);
				}

off_t	unistd.h	lseek(int fd,
		off_t offset,
		int whence)
			A *fd* által azonosított fájl "kurzorát" helyezi át *whence*-hez képest *offset* bátjnyival jobbra. A *whence* SEEK_SET (fájl eleje), SEEK_CUR (jelenlegi hely), SEEK_END (fájl vége) értékeket, míg *offset* tetszőleges előjelű egész számértéket kaphat. A "**kurzor" új pozíciójával tér vissza** (hanyadik bájtnál áll), ha nem történt hiba.	int fd = open("myfile.txt", O_RDONLY);
				off_t file_length;
				char *files_contents;
				if(fd >= 0){
				file_length = lseek(fd, 0, SEEK_END);
				if(file_length > 0){
				files_contents = malloc(file_length);
				if(files_contents){
				lseek(fd, 0, SEEK_SET);
				ssize_t remaining = file_length;
				char *write_cur = files_contents;
				while(remaining > 0){
				ssize_t temp_ret = read(fd, write_cur, remaining);
				if(temp_ret <= 0){
				break;
				}
				write_cur += temp_ret; //**write_cur nem véletlenül char***
				remaining -= temp_ret;
				}
				if(remaining == 0){

				printf("👍\n");
				}
				free(files_contents);
				}
				}
				close(fd);
				}

Folyamatkezelés

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

unsigned int	unistd.h	sleep(
		unsigned int seconds)
			seconds másodpercet vár. Ha várakozás közben jön egy signal, a handler lefutása után már nem vár tovább, visszaadja, hány másodpercet kellett volna még várnia.	for(int i = 3; i > 0; i--){
				printf("%d\n", i);
				sleep(1);
				}
				printf("exit\n");

pid_t	unistd.h	fork()	Leklónozza az őt meghívó folyamatot. Meghívásakor az operációs rendszer létrehoz egy másolatot, melyben – ahogy az eredeti folyamatban is – a futás a fork visszatérésével folytatódik. A két folyamat szinte az összes tulajdonságában megegyezik, az egyedi folyamatazonosítójuk (Process IDentifier / PID) viszont el fog térni. A **szülőben az új folyamat PID**-je lesz a visszatérési érték, míg a **gyerekben 0**. Hiba esetén negatív értékkel tér vissza.	pid_t fork_ret = **fork**();
				if(fork_ret < 0){
				printf("fork error");
				return;
				}
				printf("both processes print this\n");
				if(fork_ret){ //fork_ret != 0 (azaz fork_ret > 0 itt)
				//parent
				printf("only parent\n");
				}else{
				//child
				printf("only child\n");
				}

pid_t	sys/wait.h	wait(int* wstatus)	Addig blokkolja (nem engedi tovább futni) az őt meghívó folyamatot, míg **valamelyik gyermek** folyamata **be nem záródik**. A bezáródott gyermek folyamat azonosítójával tér vissza	int status;
				pid_t fork_ret = fork();
				if(fork_ret < 0){
				printf("fork error\n");
				return;

			vagy hiba estén -1-gyel. A folyamat befejeződésének körülményeiről tartalmaz információt a *wstatus* által mutatott szám – kivéve, ha helyette NULL pointert adtunk át, ami szintén elfogadott.	}
				if(!fork_ret){
				  sleep(5);
				  return;
				}
				pid_t term_pid = **wait**(&status);
				if(term_pid == -1){
				  printf("wait error\n");
				  //valószínűleg ekkor nincs mire várni
				}else{
				  printf("reason of termination: ");
				  if(WIFEXITED(status)){
				  printf("invoked exit() or returned from main()\t");
				  printf("returned %d\n", WEXITSTATUS(status));
				  }else if(WIFSIGNALED(status)){
				  printf("killed by signal %d\n", WTERMSIG(status));
				  }
				}

pid_t	sys/wait.h	waitpid(pid_t pid,
		  int* wstatus,
		  int options)
			Csak a ***pid* azonosítójú gyerek** folyamat bezáródását figyeli, ha viszont -1-et adunk meg, akkor a wait-hez hasonlóan mindegyik gyereket. Működését az *options* paraméterrel lehet befolyásolni, ez az érték is maradhat 0.	pid_t fork_ret1 = fork();
				if(fork_ret1 < 0){
				  printf("fork fail\n");
				  return;
				}else if(fork_ret1){
				  pid_t fork_ret2 = fork();
				  if(fork_ret2 < 0){
				  printf("fork fail\n");
				  }else if(fork_ret2){
				  //parent
				  waitpid(fork_ret2, 0, 0);
				  printf("child2 finished\n");
				  //child2-t várjuk be
				  }else{
				  //child2
				  sleep(5);
				  printf("child2 exiting\n");
				  }
				}else{

				//child1
				sleep(1);
				printf("child1 exiting\n");
				}

pid_t	unistd.h	getpid()	Visszaadja az őt meghívó folyamat egyedi azonosítóját.	pid_t fork_ret = fork();
				if(fork_ret < 0){
				printf("fork error");
				return;
				}
				if(fork_ret > 0){ //parent
				printf("PID of the child: %d\n", fork_ret);
				}else{ //child
				printf("the child's PID: %d\n", **getpid**());
				} //a két folyamat ugyanazt az azonosítót írja ki

pid_t	unistd.h	getppid()	Visszaadja a szülőfolyamat egyedi azonosítóját. Ha az eredeti szülőfolyamat már bezáródott, az árva gyerekfolyamatot a rendszerindító init program (általában PID=1 alatt fut) fogadja örökbe.	pid_t fork_ret = fork();
				if(fork_ret < 0){
				printf("fork error");
				return;
				}
				if(fork_ret > 0){ //parent
				sleep(3);
				printf("PID of the parent: %d\n", getpid());
				}else{ //child
				printf("the parent's PID: %d\n", **getppid**());
				} //itt is ugyanaz lesz a két azonosító

Signal

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

sighandler_t	signal.h	signal(int signum,
		sighandler_t handler)
			Beállítja, hogy *signum* jelzés fogadásakor mi történjen. A *handler* paraméterben az alábbiak közül valamelyiket adhatjuk meg:	int patience = 3;
				void handler(int receivedsignal){
				if(!--patience){
				exit(0);
				}

			• egy visszatérési érték nélküli, egyetlen számot fogadó függvény pointere (ez a sighandler_t típus), ami signum érkezésekor fut le, pl.: void handler(int receivedsignal){...}
			• SIG_IGN: signum érkezésekor nem történik semmi, figyelmen kívül lesz hagyva
			• SIG_DFL: visszaállítja az **alapértelmezett** működést az érkezésre, általában 3a **program kilépését**
			Visszatér a régi handlerrel, azzal, ami lekezelte volna a *signum* jelzést. A SIGKILL és SIGSTOP jelzéseket nem lehet lekezelni.
				printf("received signal no. %d (press %d more time(s) to exit)\n", receivedsignal, patience);
				}
				int main(){
				signal(SIGINT, handler);
				//ha handler helyett SIG_IGN-t használnánk, a folyamat
				//"immunis" lenne a SIGINT jelzésekre (CTRL+C-re)
				for(;;){
				sleep(1); //végtelen ciklus
				}
				//le lehet állítani a folyamatot a CTRL+C többszöri lenyomásával
				return 0;
				}

int	signal.h	kill(pid_t pid, int sig)	A *pid* azonosítójú folyamatnak egy *sig* számú jelzést küld.	**kill**(getpid(), SIGSEGV); //a program elszáll szegmentálási hibával
int	signal.h	raise(int sig)	Az őt meghívó folyamatnak küld egy *sig* jelzést.	**raise**(SIGTERM); //ekvivalens ezzel: kill(getpid(), SIGTERM)

unsigned int	signal.h	alarm(
		unsigned int seconds)
			seconds másodperc múlva egy SIGALRM jelzést küld az őt meghívó folyamatnak, de nem blokkol, meghívása után a program fut tovább. Ha már korábban be volt ütemezve jelzés, akkor azt kitörli, majd visszaadja, hogy hány másodperc múlva jött volna.	**alarm**(10);
				for(;;){
				sleep(1);
				}
				//10 másodperc múlva a SIGALRM kilövi a végtelen ciklust

int	sys/time.h	setitimer(int which,
		struct itimerval* new_value,
		struct itimerval* old_value)
			Egy új időzítőt állít be, a *new_value* által mutatott struct itimervalban megadottak szerint. Az időzítő először „it_value” után, majd „it_interval”-onként küld egy jelzést. Az idő számítását a *which* paraméterben adjuk meg:	int count = 0;
				struct itimerval ticker;
				void tick_handler(int signum){
				count++;
				printf("The program was alarmed by setitimer for the %d. time\n", count);
				if (count==5) {
				ticker.it_interval = (struct timeval){.tv_sec = 0, .tv_usec = 0};

			• **ITIMER_REAL**: a valós idő szerint mér, **SIGALRM**-ot küld
			• ITIMER_VIRTUAL: a folyamat által elhasznált processzoridőt méri, SIGVTALRM-ot küld
			• ITIMER_PROF: a folyamat által elhasznált processzoridőt, valamint azt az időt is méri, amennyit az operációs rendszer foglalkozott a folyamat kiszolgálásával, SIGPROF-ot küld
			A régebben beállított időzítőt *old_value*-ban tárolja le, amennyiben nem NULL pointer. Hiba estén -1-gyel, egyébként 0-val tér vissza.
				//ezzel iktatjuk ki az időzítőt
				raise(SIGKILL);
				//majd kilőjük a folyamatot
				}
				}
				int main(){
				ticker = (struct itimerval){
				.it_value = (struct timeval){
				.tv_sec = 5, .tv_usec = 0
				}, //kezdeti késleltetés
				.it_interval = (struct timeval){
				.tv_sec = 1, .tv_usec = 0
				} //periodikus késleltetés utána
				};
				signal(SIGALRM, tick_handler);
				setitimer(ITIMER_REAL, &ticker, 0);
				for(;;){
				sleep(1);
				}
				return 0;
				}

int	signal.h	sigemptyset(sigset_t* set)	Kiürít egy halmazt, amiben jelzéseket tudunk tárolni. Hiba estén -1-gyel tér vissza (ha NULL pointert kapott), egyébként 0-val.	void handler(int signumber){
				printf("received signal no. %d\n", signumber);
				}
				void advanced_handler(int signumber, siginfo_t *info, void* unused){
				printf("received signal no. %d from PID %d, additional value: %d\n", signumber, info->si_pid, info->si_value.sival_int);
				//vagy info->si_value.sival_ptr
				}
				int main(){
				struct sigaction sigact_term = {
				.sa_handler = handler, //de lehet SIG_DFL és SIG_IGN
				.sa_flags = 0 //ugyanúgy működjön, mint a sima signal(...)
				};
				sigemptyset(&sigact_term.sa_mask);
				sigaction(SIGTERM,& sigact_term,NULL);

int	signal.h	sigaction(int signum, struct sigaction *act, struct sigaction *old_act)	Átállítja, hogy mi történjen *signum* jelzés fogadásakor. Az *act* által mutatott sigaction structban lehet beállítani a fogadás kezelését, míg az *old_act* által mutatott fogja megkapni az előtte beállítottat – ennek a megadása nem kötelező, lehet NULL. Hiba esetén -1-gyel, egyébként 0-val tér vissza.	
int	signal.h	sigqueue(pid_t pid int sig, union sigval value)	Küld egy *sig* jelzést, valamint vele együtt egy sigval uniont is a *pid* azonosítójú folyamatnak.	

				raise(SIGTERM);
				
				struct sigaction sigact_usr1 = {
				.sa_sigaction = advanced_handler, //**nem sa_handler**
				.sa_flags = SA_SIGINFO //ez már többet tud
				};
				sigemptyset(&sigact_usr1.sa_mask);
				sigaction(SIGUSR1,& sigact_usr1,NULL);
				union sigval s_value_ptr = {.sival_int = 37}; //vagy .sival_ptr
				sigqueue(getpid(),SIGUSR1,s_value_ptr);
				return 0;
				}

int	signal.h	sigfillset(sigset_t* set)	Feltölt egy halmazt az összes jelzéssel. Hiba estén -1-gyel tér vissza (ha NULL pointert kapott), egyébként 0-val.	void dummy_handler(int signum){}
				int main(){
				pid_t fork_ret = fork();
				if(fork_ret < 0){
				printf("fork error\n");
				return 1;
				}
				if(fork_ret){
				sleep(2);
				kill(fork_ret, SIGTERM);
				printf("SIGTERM sent\n");
				sleep(2);
				kill(fork_ret, SIGUSR1);
				printf("SIGUSR1 sent\n");
				}else{
				signal(SIGUSR1, dummy_handler);
				signal(SIGUSR2, dummy_handler);
				signal(SIGTERM, dummy_handler);
				sigset_t sigset;
				sigfillset(&sigset);
				sigdelset(&sigset,SIGUSR1);
				sigdelset(&sigset,SIGUSR2);
				sigsuspend(&sigset);
				printf("finally got a SIGUSR1 or SIGUSR2\n");
				}

int	signal.h	sigdelset(sigset_t* set, int signum)	Kiszedi a *signum* jelzést a *set* halmazból. Hiba esetén -1-gyel, egyébként 0-val tér vissza.	
int	signal.h	sigsuspend(sigset_t* mask)	A *mask* halmazból kiszedett jelzésekre vár. Az őt meghívó folyamat más jelzéseket ilyenkor nem fogad. -1-gyel tér vissza.	

				return 0; }

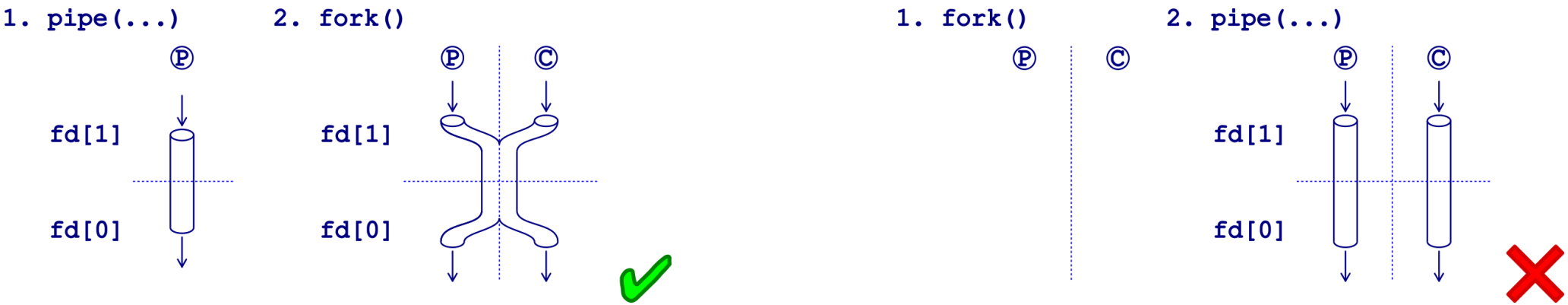
Pipe

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

int	unistd.h	pipe(int pipefd[2])	Létrehoz egy névtelen csövet: beállít két fájlleírót, amik ugyanazon buffert kezelik (az elsőn keresztül csak olvasni, a másodikon csak írni lehet). Automatikusan bezáródik, ha már nincs olyan folyamat, mely még bármelyik fájlleírót nyitva tartja. Hiba esetén -1-gyel, egyébként 0-val tér vissza. **A fork() előtt kell meghívni!**	int pipefd[2]; //**a 0-s fd csak olvasásra**
				//**az 1-es csak írásra való**
				if (**pipe**(pipefd)){
				printf("pipe fail\n");
				return;
				}
				pid_t fork_ret = fork();
				if (fork_ret < 0){
				printf("fork fail\n");
				return;
				}
				if (fork_ret){ // parent
				close(pipefd[1]);
				unsigned int strings_length;
				size_t read_remaining = sizeof(unsigned int);
				while (read_remaining){
				ssize_t read_temp = read(pipefd[0],
				((char *)&strings_length) + sizeof(unsigned int) - read_remaining, read_remaining);
				if (read_temp <= 0){
				return;
				}
				read_remaining -= read_temp;
				}
				char* buffer_to_store_string = malloc(strings_length);
				if(!buffer_to_store_string){
				return;
				}

```
  printf("going to read %d bytes from the pipe...\n",
strings_length);
  read_remaining = strings_length;
  while (read_remaining){
    ssize_t read_temp = read(pipefd[0],
                    buffer_to_store_string + strings_length -
read_remaining, read_remaining);
    if (read_temp <= 0){
      return;
    }
    read_remaining -= read_temp;
  }
  printf("got (hopefully NULL terminated) string: %s\n",
buffer_to_store_string);
  free(buffer_to_store_string);
  close(pipefd[0]);
 }else{ // child
  close(pipefd[0]);
  char *to_send = "Squeamish Ossifrage";
  unsigned int strings_length = strlen(to_send) + 1;
  size_t write_remaining = sizeof(unsigned int);
  while (write_remaining){
    ssize_t write_temp = write(pipefd[1],
                    ((char *)&strings_length) + sizeof(unsigned int) -
write_remaining, write_remaining);
    if (write_temp < 0){
      kill(getppid(), SIGKILL);
      return;
    }
    write_remaining -= write_temp;
  }
  write_remaining = strings_length;
  while (write_remaining){
    ssize_t write_temp = write(pipefd[1],
                    to_send + strings_length - write_remaining,
write_remaining);
    if (write_temp < 0){
```

				kill(getppid(), SIGKILL);
				return;
				}
				write_remaining -= write_temp;
				}
				close(pipefd[1]);
				}
				return;



Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

int	sys/types.h
	sys/stat.h
		mkfifo(char* pathname, mode_t mode)	Létrehoz egy nevesített csövet a *pathname*-ben megadott elérési úton, a *mode*-ban megadott jogosultságokkal. Ennél a változatnál a folyamatoknak maguknak kell megnyitniuk a létrehozott fájlt csak írásra, illetve csak olvasásra. Hiba estén -1-gyel, egyébként 0-val tér vissza.	char pipe_path [200];
				sprintf(pipe_path, "/tmp/unique_%d", getpid());
				if(**mkfifo**(pipe_path, S_IRUSR|S_IWUSR) == -1){
				perror("An error occurred: ");
				return;
				}
				pid_t fork_ret = fork();
				if(fork_ret > 0){ //parent
				int fd = open(pipe_path, O_RDONLY);
				//ha az egyik folyamat hamarabb hívja meg az open-t, addig
				//lesz blokkolva, amíg a cső másik felét meg nem megnyitják
				if(fd != -1){
				char msg_receive[16];
				if(read(fd, msg_receive, sizeof(msg_receive)) <= 0){
				printf("read fail\n");

int	unistd.h	unlink(char* pathname)	A *pathname* elérési útvonalon tárolt nevesített cső fájlt törli ki.	

				`}else{`
				`  printf("%s\n", msg_receive); //nem a legszebb megoldás, ide egy ciklus kellene`
				`}`
				`close(fd);`
				`}`
				`}else if(fork_ret == 0){ //child`
				`int fd = open(pipe_path, O_WRONLY);`
				`if(fd != -1){`
				`char *msg_send = "Hajrá, Fradi!";`
				`ssize_t to_send_bytes = strlen(msg_send) + 1;`
				`if(write(fd, msg_send, to_send_bytes) < to_send_bytes){`
				`//"writes of less than PIPE_BUF bytes must be atomic"`
				`//"PIPE_BUF to be at least 512 bytes"`
				`printf("write fail\n");`
				`}`
				`close(fd);`
				`}`
				`}`
				`if(fork_ret <= 0){`
				`**unlink**(pipe_path);`
				`}`

Miscellaneous

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

key_t	sys/ipc.h	ftok(char* pathname, int proj_id)	Létrehoz egy System V-s kulcsot a *pathname*-ben megadott fájl és a *proj_id* számérték segítségével. Ezt a kulcsot lehet használni később **üzenetsorok**, **szemaforok** és **osztott memória** létrehozására. A fájlnak léteznie kell és elérhetőnek kell lennie. Hiba estén -1-gyel, egyébként a kulccsal tér vissza.	`int main(int argc, char* argv[]){`
				`key_t key = **ftok**(argv[0], 37);`
				`if(key == -1){`
				`perror("ftok error: ");`
				`return 1;`
				`}`
				`return 0;`
				`}`

int	stdlib.h	rand()	Egy random számmal tér vissza a [0, RAND_MAX] intervallumról.	**srand**((unsigned int)**time**(NULL));
				//egyedi a programindulás időpontja miatt
				int double_digit = (**rand**() % 90) + 10;
				//[0, 89] + 10 => [10, 99]
				
				int least = 10, most = 99;
				int in_range = (**rand**() % (most - least + 1)) + least;

void	stdlib.h	srand(unsigned int seed)	Beállíthatjuk vele a *seed*-et, amiből rand generálja a számait. Ugyanazzal a *seed* értékkel ugyanazt a véletlen számsorozatot tudjuk generálni.	
time_t	time.h	time(time_t *tloc)	Visszatér a rendszeróra szerinti UNIX timestamppel, vagyis az 1970 január elseje éjfél óta eltelt másodpercek számával. Ha *tloc* nem NULL, letárolja az eredményt abban a változóban is.	
int	stdlib.h	atoi(char* string)	Egy egész számot tároló szövegből olvassa ki az értéket.	char* parse_this = "-90"; int parsed = atoi(parse_this);

System V message queue

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

int	sys/msg.h	msgget(key_t key,
		int msgflg)
			Létrehoz egy üzenetsort a *key*-ben megadott kulcsból. Létező üzenetsort is meg lehet vele nyitni, ha a megadott kulcshoz már tartozik egy, de újat is készíthet, amennyiben IPC_CREAT szerepel *msgflg*-ben. A *msgflg* paraméterben kell megadni továbbá, hogy ki milyen jogosultságokat kap hozzá, ami a példában oktális számrendszerrel lett megadva. Hiba estén -1-gyel, egyébként az üzenetsor azonosítójával tér vissza.	int main(int argc, char* argv[]){
				key_t key = ftok(argv[0],37);
				if(key == -1){
				perror("ftok error: ");
				return 1;
				}
				int msgQ = **msgget**(key, 0600 | IPC_CREAT);
				//0600 oktális => 6*8*8 + 0*8 + 0*1
				//**1 1** 0 0 0 0 0 0 0
				//**RW**X RWX RWX
				if(msgQ == -1){
				perror("couldn't create msgq: ");
				return 1;
				}
				if(**msgctl**(msgQ, IPC_RMID, NULL) == -1){
				perror("msgctl error: ");
				return 1;

int	sys/msg.h	msgctl(int msqid, int cmd, struct msqid_ds *buf)	A *msqid*-vel azonosított üzenetsor tulajdonságait módosítja. A bezárására használjuk, ennél a parancsnál (*cmd* = IPC_RMID),	

			alapból 0-val, hiba esetén -1-gyel tér vissza.	} return 0; }

int	sys/types.h
	sys/ipc.h
	sys/msg.h
		msgsnd(int msqid,
		void* msgp,
		size_t msgsz,
		int msgflg)
			msgp által mutatott - kötelezően long adattaggal kezdődő - structot küld át a *msqid* üzenetsoron. Az üzenet típusát a struct első adattagjában kell beállítani, a méretét *msgsz*-ben kell megadni, míg *msgflg*-ben lehet átállítani, hogy blokkoljon-e a hívás abban az esetben, ha tele lenne az üzenetsor (a 0 fog, IPC_NOWAIT nem). Ha a hívása közben jön egy jelzés, újra meg kell hívni. Alapból 0-val, hiba esetén -1-gyel tér vissza.	struct msg{
				long type;
				char buf[64];
				};
				int main(int argc, char* argv[]){
				key_t key = ftok(argv[0],37);
				if(key == -1){
				perror("ftok error: ");
				return 1;
				}
				int msgQ = msgget(key, 0600 | IPC_CREAT);
				if(msgQ == -1){
				perror("couldn't create msgq: ");
				return 1;
				}
				pid_t fork_ret = fork();
				if(fork_ret > 0){ //parent
				struct msg to_receive;
				if(**msgrcv**(msgQ, &to_receive, sizeof(to_receive) - sizeof(long), **5**, 0) == -1){
				perror("msgrcv error: ");
				}else{
				printf("got message type %d: %s\n", to_receive.type, to_receive.buf);
				}
				if(msgctl(msgQ, IPC_RMID, NULL) == -1){
				perror("msgctl error: ");
				return 1;
				}
				}else if(fork_ret == 0){ //child
				struct msg to_send = {.type = **5**, .buf = "uzenet"};
				if(**msgsnd**(msgQ, &to_send, sizeof(to_send) - sizeof(long), 0) == -1){
				perror("msgsnd error: ")

ssize_t	sys/types.h sys/ipc.h sys/msg.h	msgrcv(int msqid, void* msgp, size_t mgsz, long msgtyp, int msgflg)	Egy *msgtyp* típusú, legfeljebb *mgsz* bájt méretű – kötelezően long adattaggal kezdődő – structot (*msgp*) olvas be a *msqid* üzenetsorról. *msgflg*-ben lehet átállítani, hogy túl nagy méretű (>*mgsz*) üzenet esetén dobja el az üzenet végét, vagy adjon inkább hibát és ne olvassa ki az üzenetet (MSG_NOERROR eldobja az üzenet végét, míg 0-nál hibát dob és errno-t E2BIG-re állítja). Ha a hívása közben jön egy jelzés, újra meg kell hívni. Alapból a beolvasott üzenet méretével, míg hiba esetén -1-gyel tér vissza.	

				} } return 0; }

POSIX message queue

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

mqd_t	fcntl.h
	(errno.h)
		mq_open(char *name,
		int oflag,
		mode_t mode,
		struct mq_attr *attr)
			Megnyit egy POSIX-es (prioritásos) üzenetsort a *name*-ben megadott névvel *oflag*-ben megadott célra (O_RDONLY csak olvas, O_WRONLY csak ír vagy O_RDWR mindkettő), de újat is készíthet, ha O_CREAT is szerepel benne. Új üzenetsor készítésekor *mode*-ban meg kell adni, hogy ki milyen jogokat kaphasson az üzenetsorra, a példában ez oktális számrendszerrel történt. Az *attr* paraméterben pedig az üzenetsor maximális méreteit tartalmazó struct memóriacímét kell átadni, ami tárolja az üzenetek maximális méretét és számát. Az üzenetsor azonosítójával, hiba esetén pedig -1-gyel tér vissza.	struct mq_attr attr = {
				.mq_maxmsg = 5, //maximum hány üzenet lehet a sorban
				.mq_msgsize = 50 //egyenként legfeljebb mekkorák
				};
				char mq_name[40] = "/this_id_should_be_unique_";
				sprintf(mq_name + strlen(mq_name), "%d", getpid());
				mqd_t priority_mq = **mq_open**(mq_name, O_RDWR|O_CREAT,0600,&attr);
				if(priority_mq == -1){
				printf("mq_open error reason code: %d\n", errno);
				return;
				}
				if(**mq_close**(priority_mq) == -1){
				printf("couldn't close message queue, reason: %d\n", errno);
				}
				if(**mq_unlink**(mq_name) == -1){
				printf("mq_unlink error reason code: %d\n", errno);
				}

int	mqueue.h	mq_close(mqd_t mqdes)	Bezárja a *mqdes* POSIX-es üzenetsort, ha a folyamatnak már nincs rá szüksége. Alapból 0-val, hiba esetén -1-gyel tér vissza.	
int	mqueue.h	mq_unlink(char *name)	Törli a *name*-ben megadott névvel azonosított POSIX-es (prioritásos) üzenetsort. Alapból 0-val, hiba esetén -1-gyel tér vissza.	
int	mqueue.h	mq_send(mqd_t mqdes, char *msg_ptr,	A *mqdes* azonosítójú POSIX-es üzenetsorra egy *msg_prio* prioritású,	#define MAX_MESSAGE_LEN 50 struct mq_attr attr = {

		size_t msg_len, unsigned msg_prio)	*msg_len* bájtos üzenetet küld ki, a *msg_ptr*-től kezdődő memóriaterületről. Ha a sor már be van telve, addig vár, míg megüresedik egy hely (kivéve, ha mq_open-nél *oflag*-ben megvolt adva O_NONBLOCK). Alapból 0-val, hiba esetén -1-gyel tér vissza.	
ssize_t	mqueue.h	mq_receive(mqd_t mqdes, char *msg_ptr, size_t msg_len, unsigned *msg_prio)	A *mqdes* POSIX-es üzenetsor legnagyobb prioritású üzenetei közül a legrégebbit olvassa be *msg_ptr* által mutatott területre legfeljebb *msg_len* bájtig. msg_len **nem lehet kisebb**, mint az üzenetsor létrehozásakor megadott *mq_msgsize* érték. A beolvasott üzenet prioritását *msg_prio*-ba menti ki, amennyiben nem NULL (ami szintén elfogadott). Ha a hívása közben jön egy jelzés, újra meg kell hívni. Alapból az üzenet hosszával, hiba esetén pedig -1-gyel tér vissza.	

```
  .mq_maxmsg = 5, //maximum hány üzenet lehet a sorban
  .mq_msgsize = MAX_MESSAGE_LEN //egyenként legfeljebb
mekkorák
 };
 char mq_name[40] = "/this_id_should_be_unique_";
 sprintf(mq_name + strlen(mq_name), "%d", getpid());
 mqd_t priority_mq = mq_open(mq_name,
O_RDWR|O_CREAT,0600,&attr);
 if(priority_mq == -1){
  printf("mq_open error reason code: %d\n", errno);
  return;
 }

 pid_t fork_ret = fork();
 if(fork_ret > 0){ //parent
  char* to_send_msg = "low priority message";
  if(mq_send(priority_mq, to_send_msg, strlen(to_send_msg) + 1,
   8) == -1){
   printf("mq_send error reason code: %d\n", errno);
   kill(fork_ret, SIGKILL);
  }else{
   //késleltethetjük a magasabb prioritású üzenetet
   //sleep(2);
   to_send_msg = "higher priority message";
   if(mq_send(priority_mq, to_send_msg, strlen(to_send_msg) +
1, 30) == -1){
    printf("mq_send error reason code: %d\n", errno);
    kill(fork_ret, SIGKILL);
   }else{
    wait(NULL);
   }
  }
  mq_close(priority_mq);
  if(mq_unlink(mq_name) == -1){
   printf("mq_unlink error reason code: %d\n", errno);
  }
 }else if(fork_ret == 0){ //child
```

				char msg_buffer[MAX_MESSAGE_LEN];
				unsigned read_priority;
				ssize_t msg_read_bytes;
				if((msg_read_bytes = **mq_receive**(priority_mq, msg_buffer, MAX_MESSAGE_LEN, &read_priority)) != -1){
				printf("message having priority %u and length of %zd read: %s\n", read_priority, msg_read_bytes, msg_buffer);
				}
				if((msg_read_bytes = **mq_receive**(priority_mq, msg_buffer, MAX_MESSAGE_LEN, &read_priority)) != -1){
				printf("message having priority %u and length of %zd read: %s\n", read_priority, msg_read_bytes, msg_buffer);
				}
				mq_close(priority_mq);
				}else{
				mq_close(priority_mq);
				if(mq_unlink(mq_name) == -1){
				printf("mq_unlink error reason code: %d\n", errno);
				}
				}

Osztott memória

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

int	sys/shm.h	shmget(key_t key,
		size_t size,
		int shmflg)
			Lefoglal egy osztott memóriaterületet a *key*-ben megadott kulcsból. Létező osztott memóriát is meg lehet vele nyitni, ha a megadott kulcshoz már tartozik egy, de újat is készíthet, amennyiben IPC_CREAT szerepel *shmflg*-ben. A *shmflg* paraméterben kell megadni továbbá, hogy ki milyen jogosultságokat kap hozzá, ami a példában makrókkal lett megadva. A lefoglalni kívánt terület méretét *size*	int main(int argc, char* argv[]){
				key_t key = ftok(argv[0],73);
				if(key == -1){
				perror("ftok error: ");
				return 1;
				}
				int shm = **shmget**(key, 100, S_IRUSR | S_IWUSR | IPC_CREAT);
				if(shm == -1){
				perror("couldn't allocate shm: ");
				return 1;
				}

			paraméterben kell átadni. Hiba estén -1-gyel, egyébként az osztott memóriaterület azonosítójával tér vissza.	if(**shmctl**(shm, IPC_RMID, NULL) == -1){
				perror("shmctl error: ");
				return 1;
				}
				return 0;
				}

int	sys/shm.h	shmctl(int shmid, int cmd, struct shmid_ds *buf)	A *shmid*-vel azonosított osztott memóriaterület tulajdonságait módosítja. A felszabadítására használjuk, ennél a parancsnál (*cmd* = IPC_RMID), alapból 0-val, hiba esetén -1-gyel tér vissza.	

void*	sys/shm.h	shmat(int shmid,
		void* shmaddr,
		int shmflg);
			Rácsatol egy pointert a *shmid*-vel azonosított osztott memóriaterületre. *shmaddr*-ben meglehetne adni, hogy milyen memóriacímre szeretnénk csatolni az osztott memóriát, de ezt inkább hagyjuk NULL-on (rendszer választ). *shmflg*-ben egyéb beállítások is megadhatók (például, hogy csak olvasni szeretnénk-e a területet: SHM_RDONLY). Hiba esetén (void*) -1-gyel, egyébként pedig a csatolt memóriacímmel tér vissza.	int main(int argc, char* argv[]){
				key_t key = ftok(argv[0],73);
				if(key == -1){
				perror("ftok error: ");
				return 1;
				}
				int shm = shmget(key, 100, S_IRUSR | S_IWUSR | IPC_CREAT);
				if(shm == -1){
				perror("couldn't allocate shm: ");
				return 1;
				}
				char* shared = **shmat**(shm,NULL,0);
				if(shared == (void*)-1){
				perror("couldn't attach shm: ");
				shmctl(shm, IPC_RMID, NULL);
				return 1;
				}
				pid_t fork_ret = fork();
				if(fork_ret > 0){ //parent
				strcpy(shared, "message from parent");
				shmdt(shared);
				wait(NULL);
				if(shmctl(shm, IPC_RMID, NULL) == -1){
				perror("shmctl error: ");
				return 1;
				}

int	sys/shm.h	shmdt(void *shmaddr)	Leválasztja a korábban shmat-el felcsatolt osztott memóriaterületet. Alapból 0-val, hiba esetén -1-gyel tér vissza.	

				}else if(fork_ret == 0){ //child
				sleep(1);
				printf("child read from shared memory: %s\n", shared);
				shmdt(shared);
				}else{
				shmdt(shared);
				if(shmctl(shm, IPC_RMID, NULL) == -1){
				perror("shmctl error: ");
				return 1;
				}
				}
				return 0;
				}

Szemafor

Visszatérési érték típusa	Header	Függvényhívás	Leírás	Példakód

int	sys/sem.h	semget(key_t key,
		int nsems,
		int semflg)
			Létrehoz *nsems* szemaforból álló csoportot a *key*-ben megadott kulcsból. Létező csoportot is le lehet kérni, ha a megadott kulcshoz már tartozik egy, de újat is készíthet, amennyiben IPC_CREAT szerepel *semflg*-ben. A *semflg* paraméterben kell megadni továbbá, hogy ki milyen jogosultságokat kap hozzá, ami a példában makrókkal lett megadva. Hiba estén -1-gyel, egyébként a szemafor csoport azonosítójával tér vissza.	int main(int argc, char* argv[]){
				key_t key = ftok(argv[0],370);
				if(key == -1){
				perror("ftok error: ");
				return 1;
				}
				int sem = **semget**(key, **1**, S_IRUSR | S_IWUSR | IPC_CREAT);
				//**1** szemafort kérünk csak
				if(sem == -1){
				perror("couldn't create sems: ");
				return 1;
				}
				if(**semctl**(sem, 0, IPC_RMID) == -1){
				perror("semctl error: ");
				return 1;
				}
				return 0;
				}

int	sys/sem.h	semctl(int semid, int semnum, int cmd, ...)	A *semid*-vel azonosított szemafor csoport *semnum* indexű tagján hajt végre *cmd* parancsot. A szemafor törlésére (*cmd* = **IPC_RMID**) és számlálójának inicializálására (*cmd* =	

			SETVAL) használjuk. Alapból 0-val, hiba esetén -1-gyel tér vissza.	

int	sys/sem.h	semop(int semid,
		struct sembuf *sops,
		size_t nsops)
			A *semid*-vel azonosított szemafor csoport valamelyik tagján hajtja végre a *sops* pointerrel mutatott struct sembuf műveletét *nsops* alkalommal. A structban lehet megadni, hogy milyen indexű szemafor (.sem_num) értékéhez mennyit (.sem_op) szeretnénk adni, illetve, hogy blokkolhasson-e a hívás (.sem_flg = 0) vagy sem (.sem_flg = IPC_NOWAIT). Alapból 0-val, hiba esetén -1-gyel tér vissza.	int main(int argc, char* argv[]){
				key_t key = ftok(argv[0],370);
				if(key == -1){
				perror("ftok error: ");
				return 1;
				}
				int sem = semget(key, 1, S_IRUSR | S_IWUSR | IPC_CREAT);
				if(sem == -1){
				perror("couldn't create sems: ");
				return 1;
				}
				if(semctl(sem, 0, SETVAL, **0**) == -1){
				//0-ra állítjuk a kezdőértékét
				perror("semctl error: ");
				return 1;
				}
				pid_t fork_ret = fork();
				if(fork_ret > 0){ //parent
				printf("first\n");
				sleep(1);
				struct sembuf muvelet = {.sem_num = 0,
				.sem_op = **1**, .sem_flg = 0};
				if (**semop**(sem, &muvelet, 1) < 0){
				perror("semaphore operation error: ");
				}
				}else if(fork_ret == 0){ //child
				struct sembuf muvelet = {.sem_num = 0,
				.sem_op = **-1**, .sem_flg = 0};
				if (**semop**(sem, &muvelet, 1) < 0){
				perror("semaphore operation error: ");
				}
				printf("second\n");
				}
				if(fork_ret <= 0){
				if(semctl(sem, 0, IPC_RMID) == -1){

				perror("semctl error: ");
				return 1;
				}
				}
				return 0;
				}

Operációs rendszerek ZH segédlet

Kiindulókód

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <signal.h>
#include <fcntl.h>
#include <time.h>
#include <mqueue.h>
#include <sys/msg.h>
#include <sys/sem.h>
#include <sys/shm.h>
#include <sys/stat.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/wait.h>

#define pt(fstr, ...) {printf("[%s] "fstr"\n", whoami ?: "N/A", ##__VA_ARGS__);}
char* whoami = 0;

/*void dummy_handler(int x){}
void do_not_kill_me(int signum){signal(signum, dummy_handler);}*/

/*void szemafor_muvelet(int semid, int sem_op){
  struct sembuf muvelet = {.sem_num = 0, .sem_op = sem_op, .sem_flg = 0};
  semop(semid, &muvelet, 1);
}*/

int main(int argc, char* argv[]){
  pid_t gy1 = fork();
  if(gy1){
    whoami = "szulo";
    pt("kilepes");
  }else{
    whoami = "gyerek";
    pt("kilepes");
  }
  return 0;
}
```

Fekete Máté